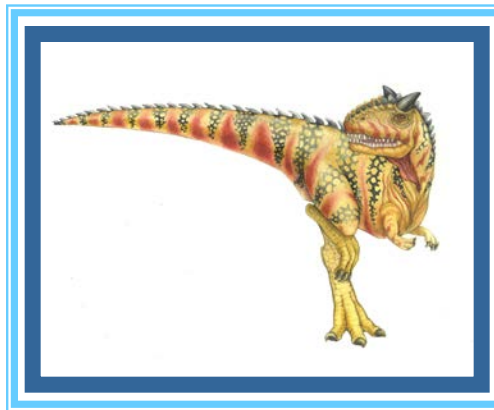


Chapter 1: Introduction





Chapter 1: Introduction

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Structure
- Operating-System Operations
- Process Management
- Memory Management
- Storage Management
- Protection and Security
- Kernel Data Structures
- Computing Environments
- Open-Source Operating Systems





Objectives

- To describe the basic organization of computer systems
- To provide a grand tour of the major components of operating systems
- To give an overview of the many types of computing environments
- To explore several open-source operating systems





What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:

The main goals of an Operating System are:

Convenience – Makes computer easier to use for the user.

Efficiency – Uses hardware resources in the best possible way.

Resource Management – Manages CPU, memory, and I/O devices effectively.

Security & Protection – Protects data and resources from unauthorized access.

Error Handling & Reliability – Detects and recovers from errors for smooth operation.





Computer System Structure

- Computer system can be divided into four components:
 - **Hardware** – provides basic computing resources
 - CPU, memory, I/O devices
 - **Operating system**
 - Controls and coordinates use of hardware among various applications and users
 - **Application programs** – These are the software that helps users do specific tasks using the computer.

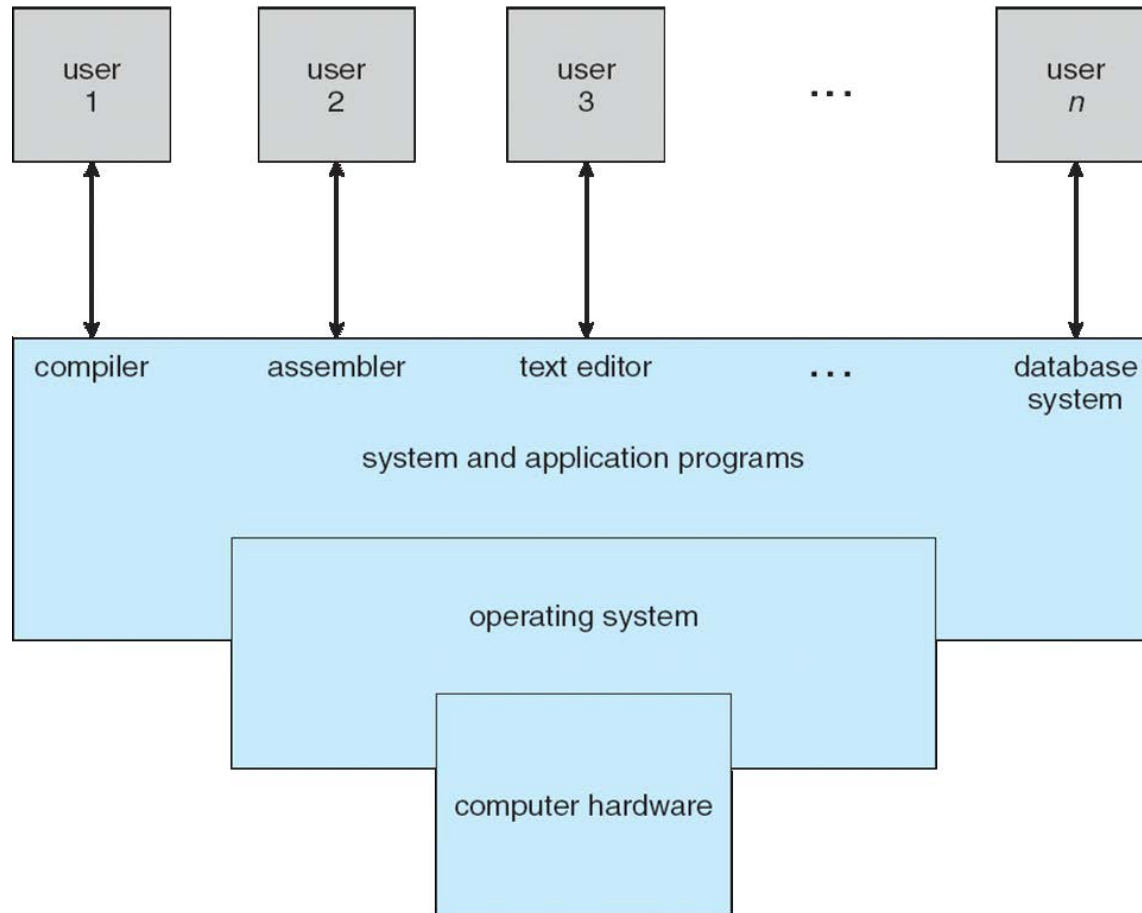
Examples:

- Word processors, compilers, web browsers, database systems, video games
- **Users**
 - People, machines, other computers





Four Components of a Computer System





What Operating Systems Do



What an operating system does depends on who is using the computer

- Normal users want the computer to be easy to use and fast.
→ They don't worry about how resources like CPU or memory are used.
- Shared computers (like big mainframe systems) must keep many users happy at the same time.
- Workstations (personal computers for work) usually have their own resources, but sometimes use shared resources from servers.
- Handheld devices (like smartphones) have limited resources.
→ They are designed to save battery and be easy to use.
- Embedded computers (inside cars or machines) often have no screen or direct user interface.
→ They just do their job in the background.





Operating System Definition



■ OS is a resource allocator

- It manages everything like CPU, memory, and devices.
- It decides who gets what, so all programs run fairly and efficiently.

■ OS is a control program

- Controls execution of programs to prevent errors and improper use of the computer

Functions of an Operating System

1. **Process Management** – Creates, schedules, and terminates processes.
2. **Memory Management** – Allocates and deallocates main memory.
3. **File Management** – Organizes, stores, and retrieves data in files.
4. **Device Management** – Controls and coordinates input/output devices.
5. **Security & Protection** – Protects data and resources from unauthorized access.





Operating System Definition (Cont.)

What is the Kernel?

The kernel is the core part of an operating system.

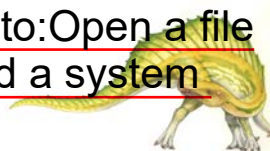
It is always running in the background while the computer is on.

It connects the hardware and the software.

What the Kernel Does:

- Manages memory
 - Decides how much memory each program gets.
- Manages CPU (Processor)
 - Decides which program runs and for how long.
- Manages devices
 - Helps programs talk to things like printers, keyboards, and hard drives
- Manages system calls
 - Lets applications request services (like opening a file).

A system call is how a program talks to the operating system. If a program wants to: Open a file It cannot do it directly. It asks the operating system to do it — this request is called a system call.





Computer Startup

What is a Bootstrap Program?

The bootstrap program is the first program that runs when we turn on or restart a computer.

What It Does:

- It is stored in ROM or EPROM → This is permanent memory (called firmware).
- It starts the computer and sets up everything (like memory, CPU, and devices).
- It loads the operating system kernel into memory.
- Then it starts the OS, and the computer is ready to use.

Firmware is a special type of software stored inside hardware devices, usually in ROM, EPROM, or Flash memory. It is permanent or semi-permanent and provides the basic instructions needed for the hardware to function correctly. Without firmware, devices like computers, phones, printers, or TVs wouldn't know how to start or operate properly

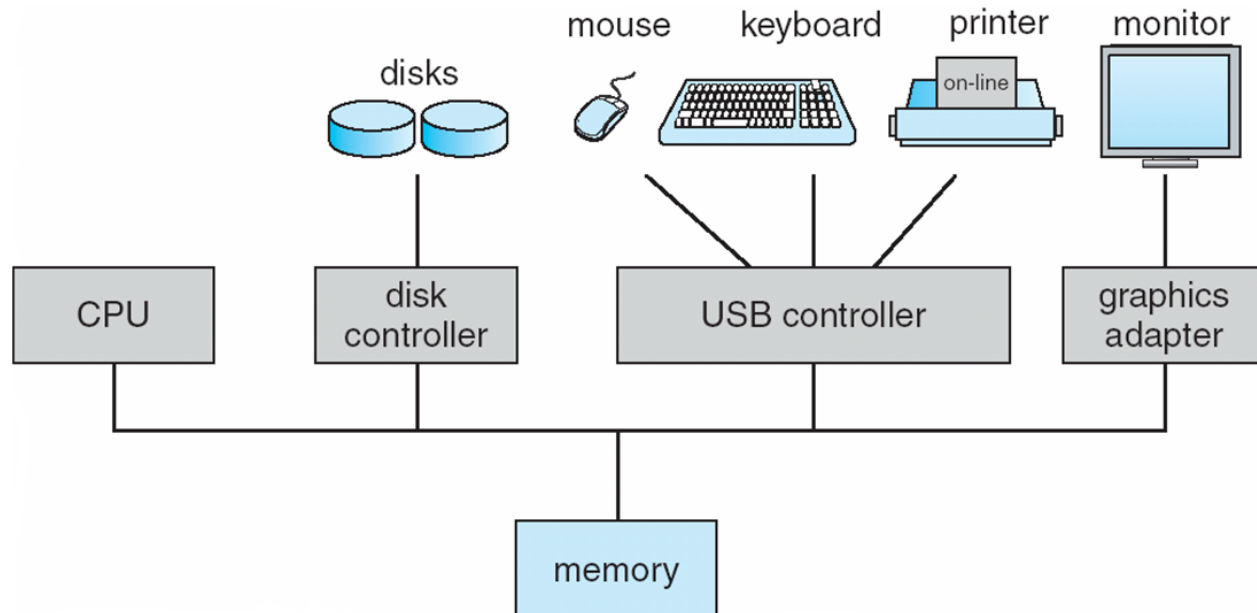




Computer System Organization

■ Computer-system operation

- One or more CPUs, device controllers connect through common bus providing access to shared memory
- Concurrent execution of CPUs and devices competing for memory cycles





Computer-System Operation

I/O Devices and CPU Work Together

CPU and I/O devices can work at the same time (concurrently).

How It Works:

- Each I/O device (like keyboard, printer, disk) has its own controller.
→ A controller is like a manager for that device.

- Each controller has a small storage called a local buffer.

- Data moves like this:

From main memory ↔ to/from local buffer

From device ↔ to/from local buffer

- After finishing its job, the device controller sends an interrupt to the CPU.

→ This tells the CPU, "I'm done!"





Common Functions of Interrupts

- When an **interrupt** happens, the CPU stops its current work and runs a small program called the **Interrupt Service Routine (ISR)**.
- The CPU uses an **interrupt vector** (a table of addresses) to find where each ISR is stored in memory.
- Before jumping to the ISR, the CPU **saves the address** of the instruction it was running, so it can **continue later** after finishing the ISR.
- A **trap (or exception)** is also like an interrupt, but it comes from **inside the program**.
 - Example: divide by zero error.
 - Example: system call (when a program asks the OS for help).
- An **interrupt** is simply a **signal to the CPU** that tells it to stop its current task and give attention to something important.
- The **operating system is interrupt-driven**, which means it responds to events (like input or errors) using interrupts instead of continuously checking everything.





Interrupt Handling

- When an interrupt happens, the operating system saves the CPU's current state.
- This means it stores things like the registers and the program counter (the place where the CPU was working).
- Then the OS needs to figure out which device or event caused the interrupt. It does this by:

Polling: Asking each device in order, "Is this your interrupt?"

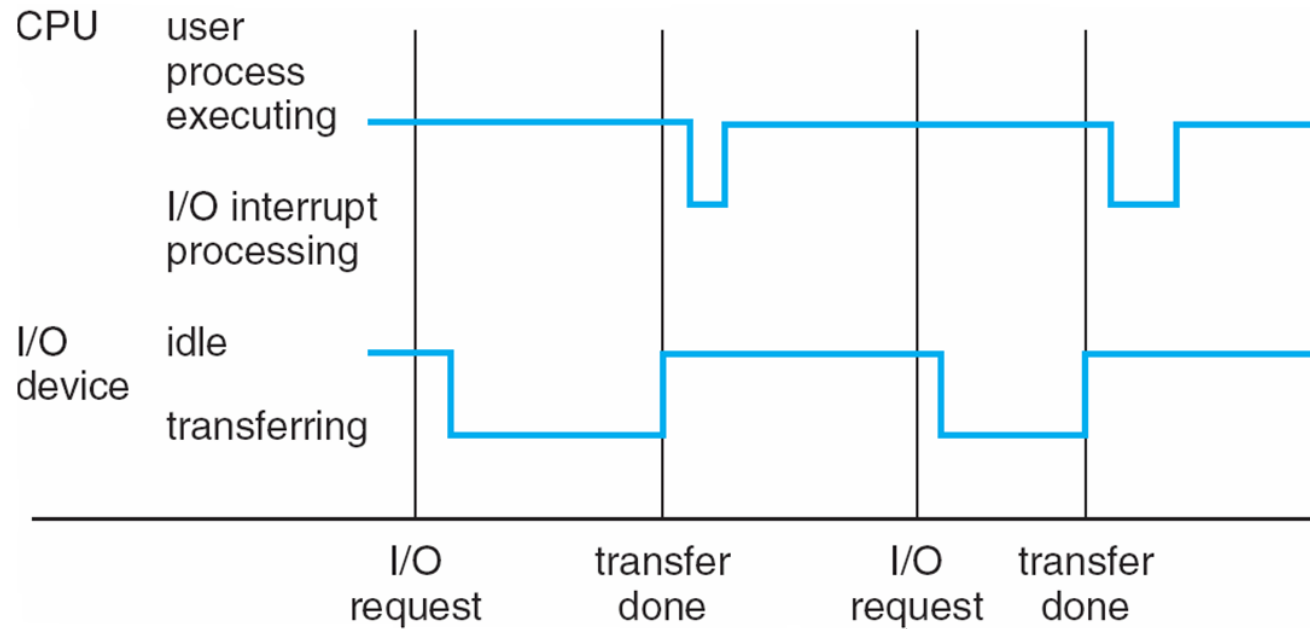
Vectored Interrupt System: The interrupt itself tells the OS exactly which device or event caused it.

- After identifying the interrupt, the OS runs a special piece of code (an interrupt handler) to take the right action for that interrupt.





Interrupt Timeline





I/O Structure

- **When the CPU waits for I/O to finish:**
 - After starting an I/O operation, the CPU waits (using a wait instruction) until the I/O is done.
 - This waiting idles the CPU, meaning it can't do other work
 - This waiting often uses a wait loop which repeatedly checks if I/O is done (which wastes memory access time).
 - Only one I/O request can be handled at a time — no multitasking of I/O
- **When the CPU doesn't wait for I/O to finish immediately:**
 - After starting I/O, the CPU immediately continues running the user program without waiting.
 - The program can use a system call to ask the OS to wait for the I/O to complete if needed.
 - The OS keeps a device-status table that stores info about each device (type, address, and current state).
 - The OS looks up this table to check device status and update it when interrupts happen.





Storage Definitions and Notation Review

The basic unit of computer storage is the **bit**. A bit can contain one of two values, 0 and 1. All other storage in a computer is based on collections of bits. Given enough bits, it is amazing how many things a computer can represent: numbers, letters, images, movies, sounds, documents, and programs, to name a few. A **byte** is 8 bits, and on most computers it is the smallest convenient chunk of storage. For example, most computers don't have an instruction to move a bit but do have one to move a byte. A less common term is **word**, which is a given computer architecture's native unit of data. A word is made up of one or more bytes. For example, a computer that has 64-bit registers and 64-bit memory addressing typically has 64-bit (8-byte) words. A computer executes many operations in its native word size rather than a byte at a time.

Computer storage, along with most computer throughput, is generally measured and manipulated in bytes and collections of bytes.

A **kilobyte**, or **KB**, is $1,024$ bytes

a **megabyte**, or **MB**, is $1,024^2$ bytes

a **gigabyte**, or **GB**, is $1,024^3$ bytes

a **terabyte**, or **TB**, is $1,024^4$ bytes

a **petabyte**, or **PB**, is $1,024^5$ bytes

Computer manufacturers often round off these numbers and say that a megabyte is 1 million bytes and a gigabyte is 1 billion bytes. Networking measurements are an exception to this general rule; they are given in bits (because networks move data a bit at a time).





Storage Structure

■ Main Memory (RAM)

- It's the only big storage that the CPU can access directly.
- It uses random access → CPU can jump to any part of memory instantly.
- It is volatile → Data is lost when the power is off.

■ Secondary Storage

- It's like an extension of main memory, but nonvolatile → Data stays even when power is off.
- It stores large amounts of data.

■ Hard Disks

- Made of metal or glass platters with magnetic material
- Data is stored in tracks, which are split into sectors.
- A disk controller handles communication between the disk and the computer.

■ Solid-State Drives (SSD)

- Faster than hard disks.
- Also nonvolatile (data stays when powered off).
- Use different modern technologies (no moving parts).
- Becoming more common due to speed and durability.





Storage Hierarchy

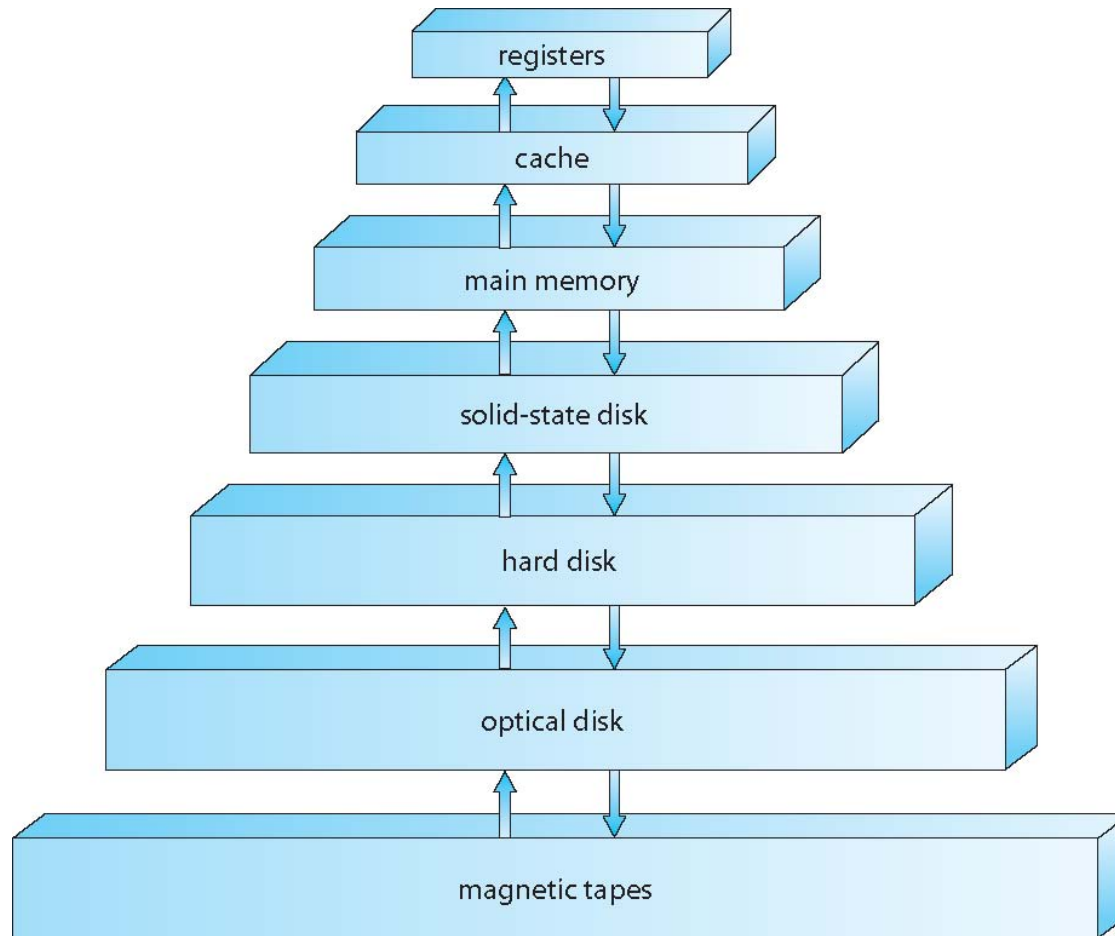
- Storage systems organized in hierarchy
 - Speed
 - Cost
 - Volatility
- **Caching** –Caching means keeping a small copy of frequently used data in a faster storage (called cache).It is used so the CPU doesn't always need to fetch data from slow main memory
- **Device Driver** A device driver is software that helps the OS talk to hardware devices (like keyboard, disk, printer).

It gives a standard way (uniform interface) for the kernel to use different devices, even if the hardware is different.





Storage-Device Hierarchy





Caching

- Caching is used at many levels in a computer:
→ In hardware, operating systems, and software.

How It Works:

- When the computer needs data, it first checks the cache (faster storage).
- If the data is in the cache → it uses it quickly.
- ☑ This is called a cache hit.
- If the data is not in the cache → it copies it from slower storage (like RAM or disk) into the cache and then uses it.
- ✗ This is called a cache miss.

■ Key Points:

- Cache is smaller than the main storage it helps with.
- Managing the cache is a big design challenge.
- Designers must decide:
 - How big the cache should be (cache size)
 - Which data to remove when the cache is full (replacement policy)





Direct Memory Access Structure

■ Direct Memory Access (DMA)

- DMA is a system that allows an I/O device to transfer data directly to or from the memory without constantly involving the CPU. This makes data transfer faster and more efficient because the CPU can do other tasks while the transfer happens.

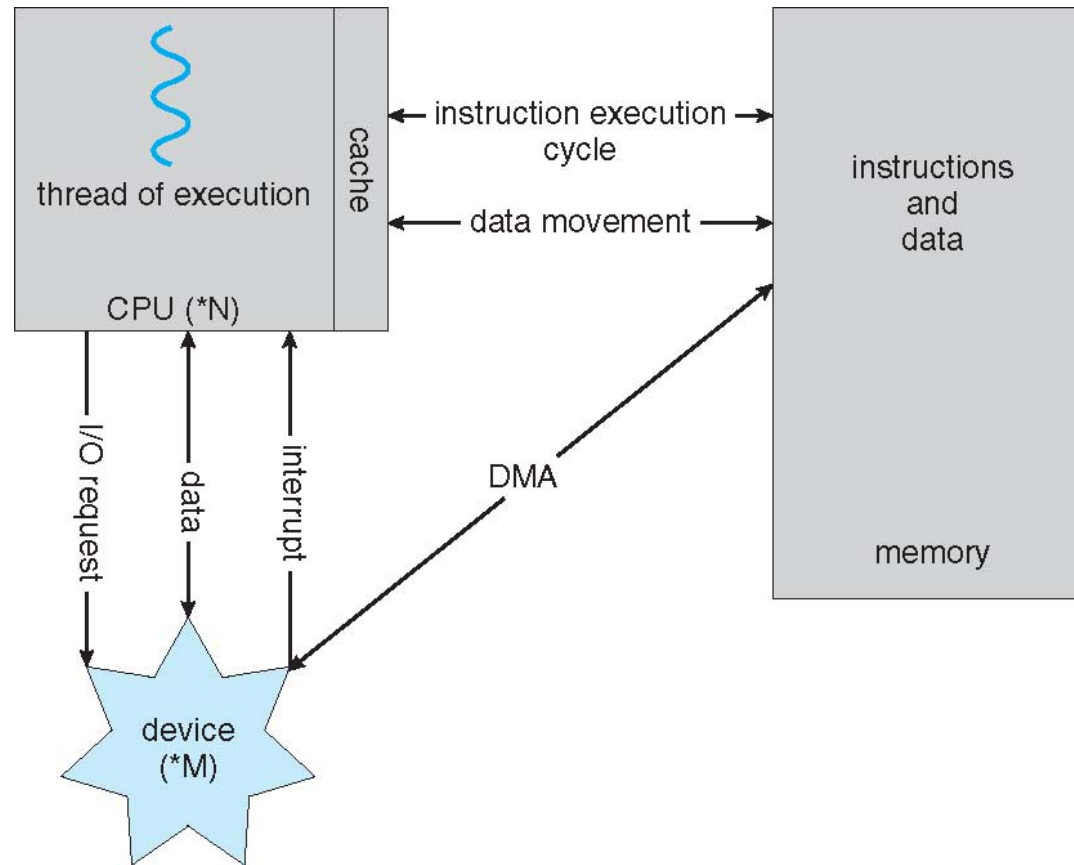
How DMA Works:

- Normally, the CPU handles data transfer between devices and memory → slow and busy.
- With DMA, the device controller sends data directly to main memory.
 - ☑ The CPU is not involved in moving each byte.
- Only one interrupt is sent to the CPU after the entire block of data is transferred.
 - ☑ This is much better than interrupting for every single byte!





How a Modern Computer Works



A von Neumann architecture





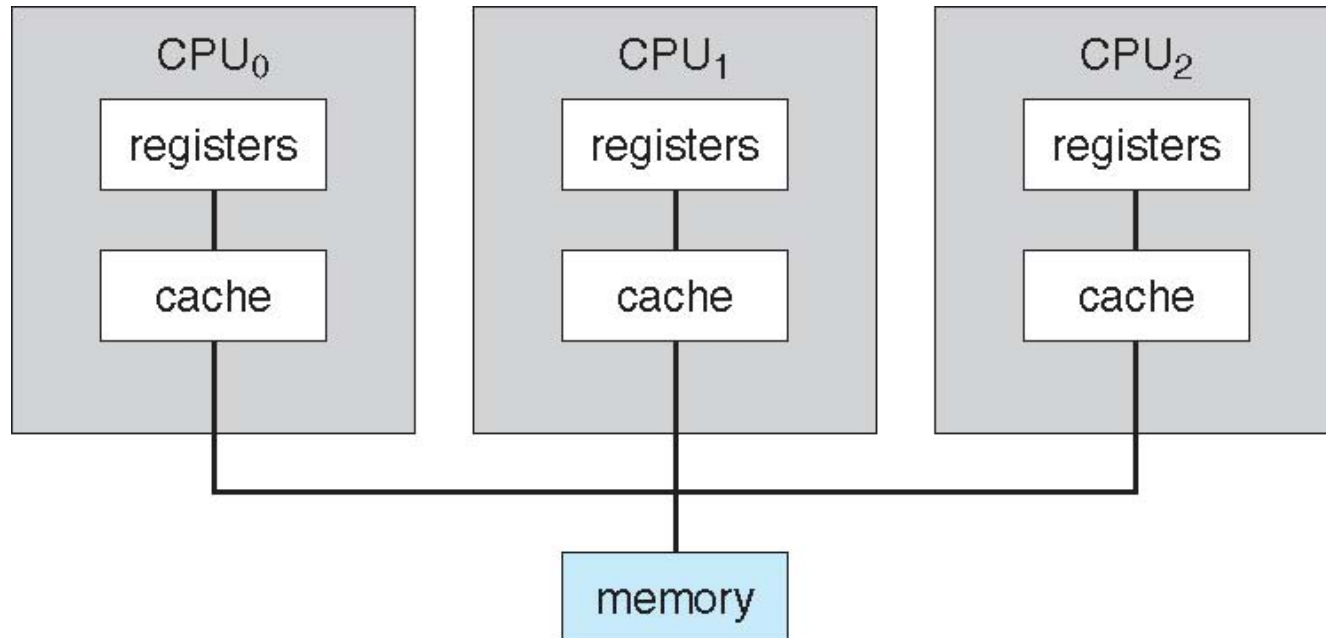
Computer-System Architecture

- Most systems use a single general-purpose processor
 - Most systems have special-purpose processors as well
- **Multiprocessors** Multiprocessor System = Multiple CPUs
 - Also known as **parallel systems**, **tightly-coupled systems**
 - Advantages include Multiprocessor Systems:
 1. **Increased throughput**-More work done in less time
 2. **Economy of scale**-Cheaper than using multiple single-CPU systems
 3. **Increased reliability**-If one CPU fails, others can still work(called fault tolerance)
 - Two types:
 1. **Asymmetric Multiprocessing** – each processor is assigned a specie task.
 2. **Symmetric Multiprocessing** – each processor performs all tasks





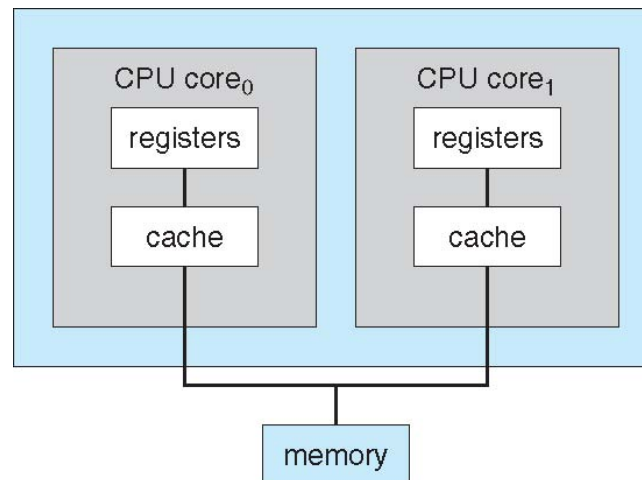
Symmetric Multiprocessing Architecture





A Dual-Core Design

- Multi-chip and **multicore**
- A dual-core processor has two processing units (cores) inside a single chip. Each core can fetch, decode, and execute instructions independently. This means the CPU can do two tasks at the same time (parallel processing).





Clustered Systems

Clustered systems are multiple connected computers that work together and share storage (usually via SAN) to improve performance, reliability, and ensure continuous service if one fails.

They provide high availability - the system keeps running even if one machine fails. Used for better performance, reliability, or both.

Types of Clustering:

Asymmetric clustering One computer does the work, while the other just waits in standby, ready to take over if the first one fails.

Symmetric clustering All computers (nodes) are active, doing work and watching each other.

Use in High-Performance Computing (HPC):

- Some clusters are used for powerful tasks like scientific research or simulations.
- These apps must be written to support parallel processing (splitting tasks across computers).

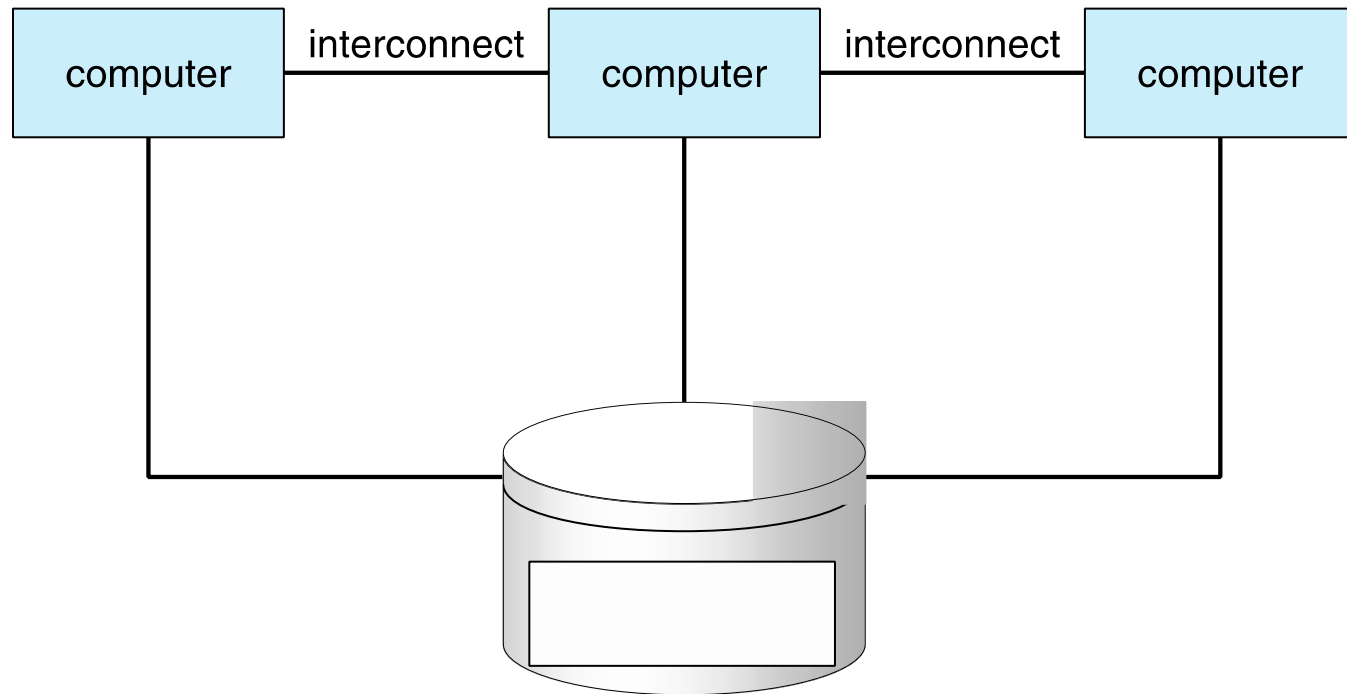
Distributed Lock Manager (DLM):

- Used to prevent conflicts when many nodes access shared resources.
- It ensures only one node changes a shared file or data at a time.





Clustered Systems





Operating System Structure

■ Multiprogramming (Batch system) needed for efficiency

Why Needed: A single user/program can't keep both CPU and I/O devices busy all the time, which leads to wasted system resources.

What It Does: Multiprogramming allows multiple jobs (programs with their code and data) to be in memory at once. It ensures the CPU always has something to execute, improving efficiency.

In Memory: Only a subset of all jobs is kept in main memory at a time

Job Scheduling: The OS chooses one job to run at a time using job scheduling.

When a Job Waits (e.g., for I/O): The OS switches to another job so that the CPU can keep working while the first job waits.

■ Timesharing (multitasking)

What It Is: A logical extension of multiprogramming. The CPU switches between many users or tasks very quickly, giving the feeling that all are running at the same time.

Interactive Computing: Users can interact with their programs while they run, making the system interactive.

Fast Response: The system aims for a response time of less than 1 second to keep it interactive.

Each User = One Process: Every user has at least one running program (called a process) in memory.



Multiple Ready Jobs: If many jobs are ready, the CPU uses scheduling to decide which one to run next.

Not Enough Memory? If memory is full, the OS uses swapping to move processes in and out of memory.

Virtual Memory: Allows programs to run even if they are not fully loaded into memory, by using disk space as temporary memory.

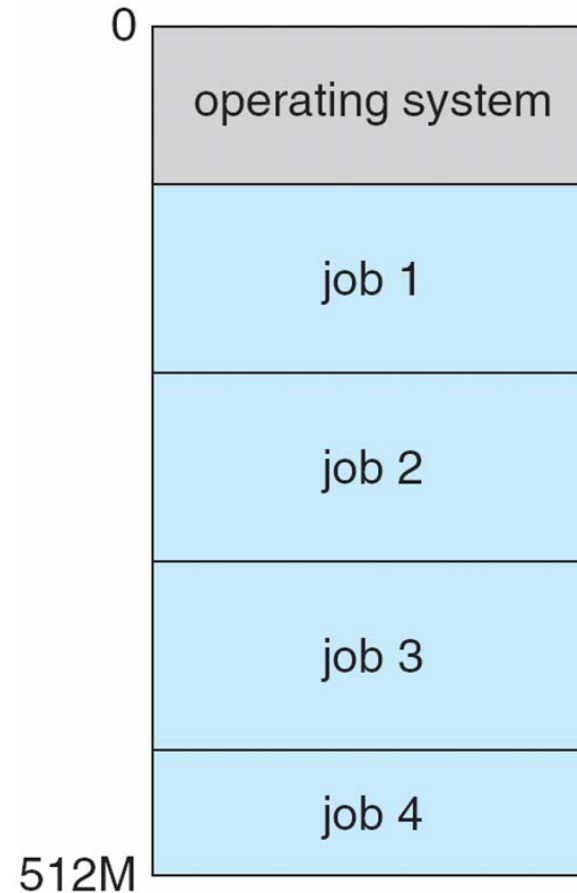
Virtual Memory

Virtual memory is a computer memory management technique that combines RAM and disk storage to create the illusion of a larger memory space than physically available

When RAM is full, the OS temporarily moves some data from RAM to a space on the hard disk (called the swap space or page file). It gives each program the illusion of having its own large, continuous memory, even if the actual physical RAM is small. The OS handles it automatically, so programs don't need to know about it.



Memory Layout for Multiprogrammed System





Operating-System Operations

- **Interrupt driven** Systems respond to events using interrupts from hardware or software
 - **Hardware interrupt** by one of the devices (e.g., keyboard, mouse).
 - **Software interrupt** (**exception** or **trap**):
 - ▶ Software error (e.g., division by zero)
 - ▶ Request for operating system service
 - ▶ Other process problems include infinite loop, processes modifying each other or the operating system





Operating-System Operations (cont.)

Dual-mode operation is a feature of modern operating systems that allows the CPU to run in two modes:

1. User Mode:

- Normal mode for running **user programs**.
- CPU has **restricted access** to critical instructions and hardware.
- Prevents programs from accidentally or maliciously **damaging the system**.

2. Kernel (or Supervisor) Mode:

- Special mode for the **operating system**.
- CPU has **full access** to all instructions and hardware.
- Used to execute **privileged instructions**, like managing memory, I/O, or interrupts.

How it Works

- When a **user program** needs OS services, it uses a **system call**, which switches the CPU from **user mode** → **kernel mode**.
- After the OS completes the request, the CPU switches **back to user mode**.

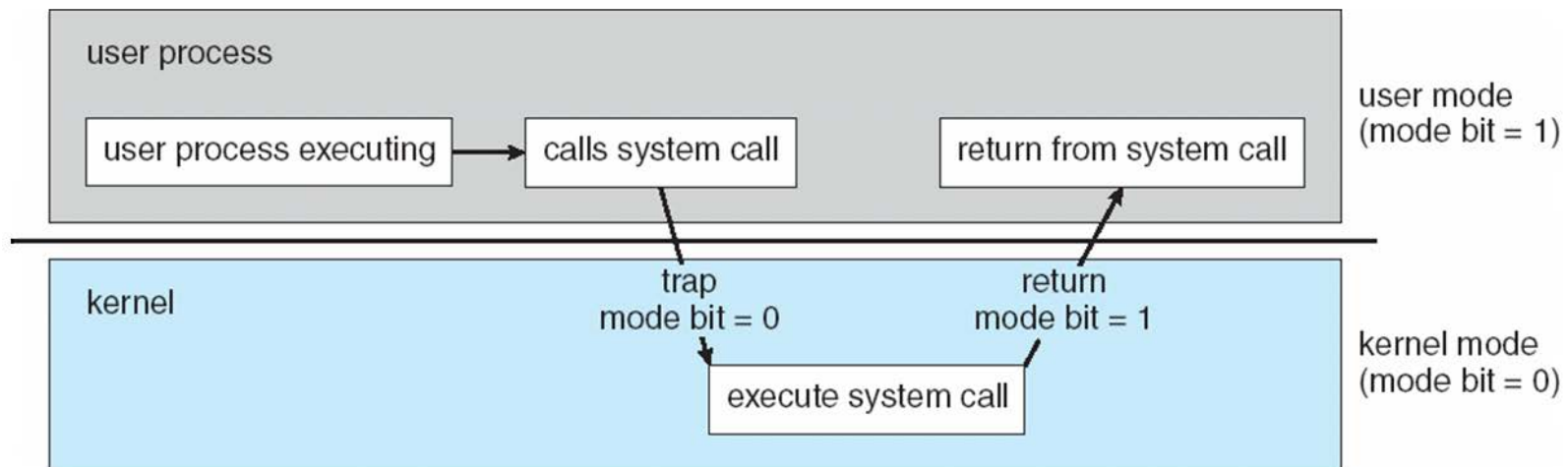




Transition from User to Kernel Mode

Timer to Prevent Infinite Loop / Process Hogging Resources

- A **timer** is used to **interrupt the CPU** after a specific time period to prevent a program from running forever or hogging resources.
- The OS keeps a **counter**, which is decremented by the **physical clock**.
- The **operating system** sets this counter using **privileged instructions**.
- When the counter reaches **zero**, the timer **generates an interrupt**.
- This mechanism is **set up before scheduling a process**, allowing the OS to **regain control** or **terminate a program** that exceeds its allotted time.





Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a **passive entity**, process is an **active entity**.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim of any reusable resources

Single-threaded Process:

- Has one program counter that keeps track of the next instruction to execute.
- Executes instructions one by one, sequentially, until it finishes.

Multi-threaded Process:

- Has one program counter per thread.
- Multiple threads in the same process can run concurrently.

Concurrency in a System:

- A system usually runs many processes at the same time (some user programs, some OS tasks).
- CPUs are shared among processes and threads using multiplexing, giving the appearance of simultaneous execution.





Process Management Activities

1. Process Creation and Termination

- Create new processes and terminate them when done.
- Example system calls: `fork()` , `exit()` .

2. Process Scheduling

- Decide **which process runs next** on the CPU.
- Types of scheduling:
 - **Long-term**: Decides which jobs enter the system.
 - **Medium-term**: Suspends or resumes processes.
 - **Short-term**: Chooses which ready process runs next.

3. Process Synchronization

- Ensure processes **cooperate safely** when sharing resources.
- Example: Producer-Consumer problem, semaphores, mutexes.

4. Inter-Process Communication (IPC)

- Allow processes to **exchange data and signals**.
- Methods: Shared memory, message passing, pipes, sockets.

5. Deadlock Handling

- Detect and handle situations where processes **wait indefinitely** for resources.





Memory Management

- Memory management is the part of the OS that controls and organizes the computer's main memory. It ensures that processes have enough memory to execute and that memory is used efficiently.
- Memory management activities
 - **Allocation of Memory:** Assign memory to processes when they need it. It can be contiguous allocation, non-contiguous allocation (paging, segmentation)
 - **Tracking Memory Usage:** Keep track of which parts of memory are free or in use.
 - **Swapping and Paging:** Move processes between main memory and secondary storage to free space. Paging divides memory into fixed-size pages; Segmentation divides memory into logical segments.
 - **Protection and Isolation:** Ensure one process cannot access memory of another process.
 - **Virtual Memory:** Makes the system appear to have more memory than physically available using techniques like demand paging.





Storage Management

- Storage management is a function of the operating system that controls how data is stored, organized, and accessed in a computer. It manages main memory (RAM), secondary storage (hard disks, SSDs), and cache so that programs can run efficiently.
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - ▶ Creating and deleting files and directories
 - ▶ Providing operations to manipulate files and directories.
 - ▶ Mapping files onto secondary storage
 - ▶ Backup files onto stable (non-volatile) storage media





Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its algorithms
- OS activities
 - Free-space management
 - Storage allocation
 - Disk scheduling
- Some storage need not be fast
 - Tertiary storage includes optical storage, magnetic tape
 - Still must be managed – by OS or applications
 - Varies between WORM (write-once, read-many-times) and RW (read-write)





Performance of Various Levels of Storage

Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

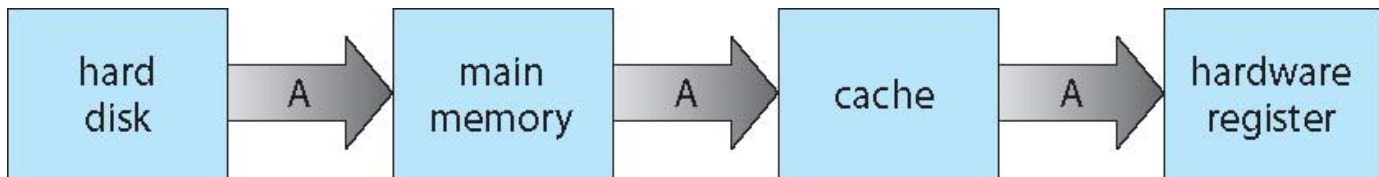
Movement between levels of storage hierarchy can be explicit or implicit





Migration of data “A” from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy



- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache
- Distributed environment situation even more complex
 - Several copies of a datum can exist
 - Various solutions covered in Chapter 17





I/O Subsystem

- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
 - General device-driver interface
 - Drivers for specific hardware devices





Protection and Security

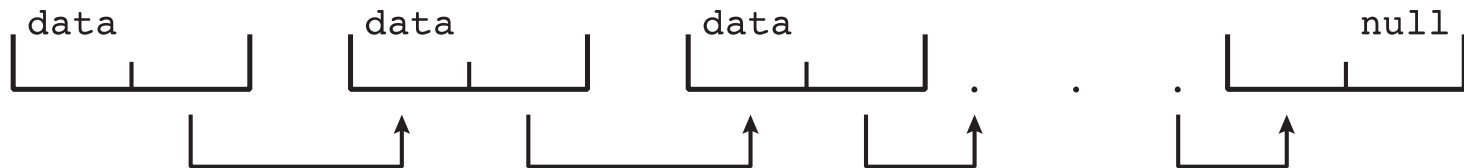
- **Protection** – A mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – Security is the set of measures used to protect the operating system and its resources from unauthorized access, misuse, or attacks.
- **User and Access Management:**
 - **User Identification:** Each user has a user ID (name + number). All files and processes are associated with this ID to control access.
 - **Groups:** Users can be organized into groups using a group ID. Access permissions can be set for the whole group.
 - **Privilege escalation:** Allows a user to temporarily change to an ID with more rights to perform certain tasks



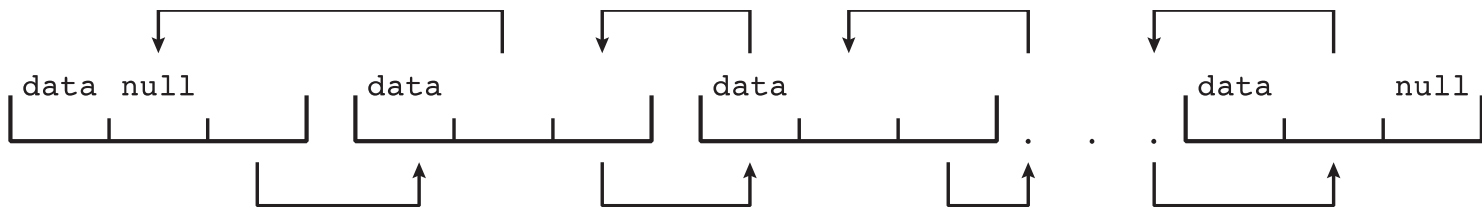


Kernel Data Structures

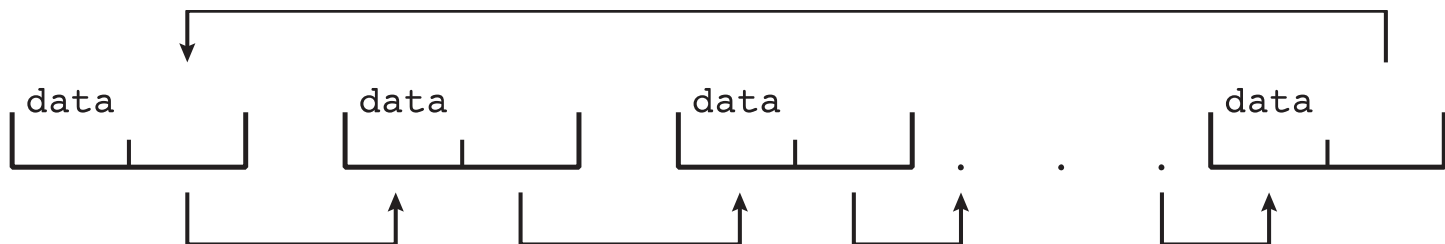
- Many similar to standard programming data structures
- ***Singly linked list***



- ***Doubly linked list***



- ***Circular linked list***



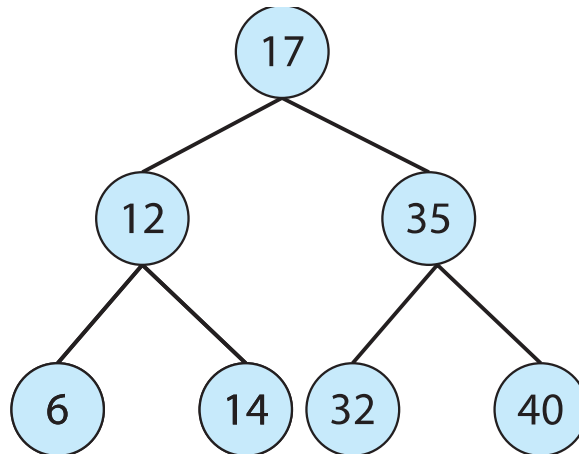


Kernel Data Structures

■ Binary search tree

left $\leq$ right

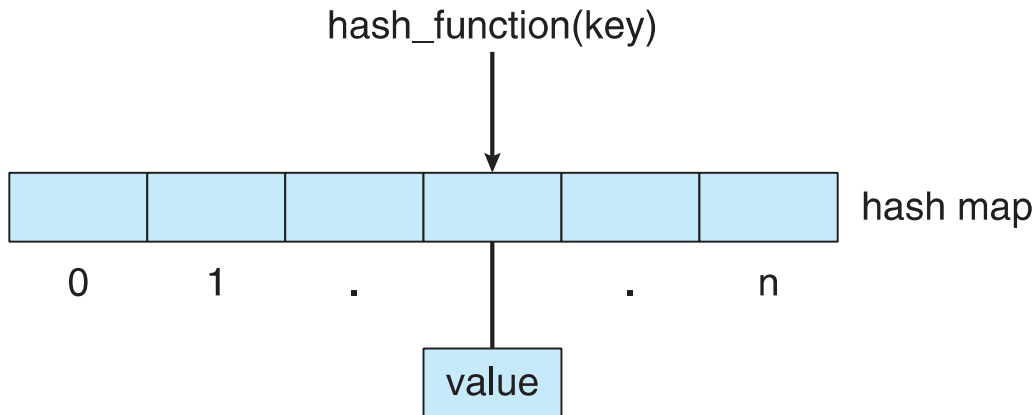
- Search performance is $O(n)$
- **Balanced binary search tree** is $O(\lg n)$





Kernel Data Structures

- **Hash function** can create a **hash map**



- **Bitmap** – string of n binary digits representing the status of n items
- Linux data structures defined in
include files `<linux/list.h>`, `<linux/kfifo.h>`,
`<linux/rbtree.h>`





Computing Environments - Traditional

- Stand-alone general purpose machines
- But blurred as most systems interconnect with others (i.e., the Internet)
- **Portals** provide web access to internal systems
- **Network computers** (**thin clients**) are like Web terminals
- Mobile computers interconnect via **wireless networks**
- Networking becoming ubiquitous – even home systems use **firewalls** to protect home computers from Internet attacks





Computing Environments - Mobile

- Handheld smartphones, tablets, etc
- What is the functional difference between them and a “traditional” laptop?
- Extra feature – more OS features (GPS, gyroscope)
- Allows new types of apps like ***augmented reality***
- Use IEEE 802.11 wireless, or cellular data networks for connectivity
- Leaders are **Apple iOS** and **Google Android**





Computing Environments – Distributed

- Distributed computing
 - Distributed computing is a system where multiple computers (nodes) work together to solve a problem or perform tasks.
 - ▶ **Network** is a communications path, **TCP/IP** most common –
 - Local Area Network (LAN)**
 - **Wide Area Network (WAN)**
 - **Metropolitan Area Network (MAN)**
 - **Personal Area Network (PAN)**
 - **Network Operating System** provides features between systems across network
 - ▶ Communication scheme allows systems to exchange messages
 - ▶ Illusion of a single system

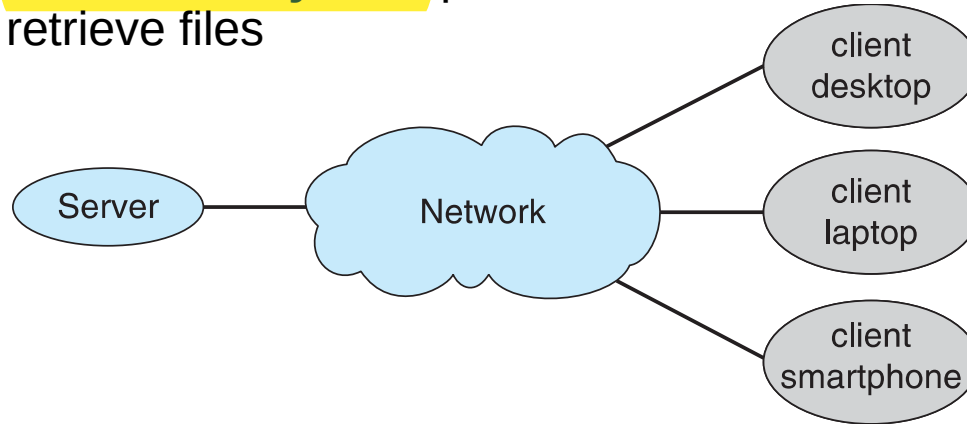




Computing Environments – Client-Server

■ Client-Server Computing

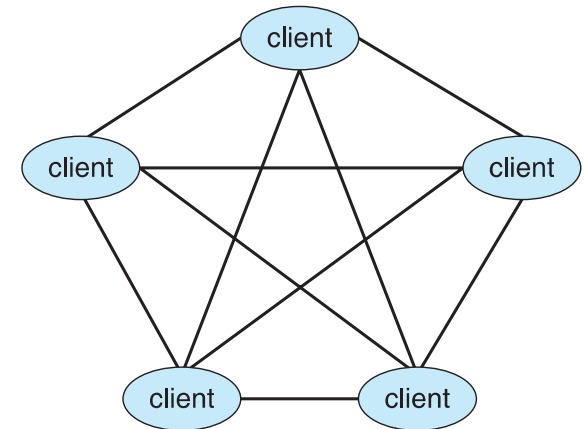
- A client-server system is a type of distributed computing where tasks are divided between clients and servers.
- Many systems now **servers**, responding to requests generated by **clients**
 - ▶ **Compute-server system** Provides an interface for clients to request services like running computations or accessing databases
 - ▶ **File-server system** provides interface for clients to store and retrieve files





Computing Environments - Peer-to-Peer

- Another model of distributed system
- P2P does not distinguish clients and servers
 - Instead all nodes are considered peers
 - May each act as client, server or both
 - Node must join P2P network
 - ▶ Registers its service with central lookup service on network, or
 - ▶ Broadcast request for service and respond to requests for service via **discovery protocol**
 - Examples include Napster and Gnutella, **Voice over IP (VoIP)** such as Skype





Computing Environments - Virtualization

- Allows operating systems to run applications within other OSES
 - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
 - Generally slowest method
 - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for CPU, running **guest** OSES also natively compiled
 - Consider VMware running WinXP guests, each running applications, all on native WinXP **host** OS
 - **VMM** (virtual machine Manager) provides virtualization services





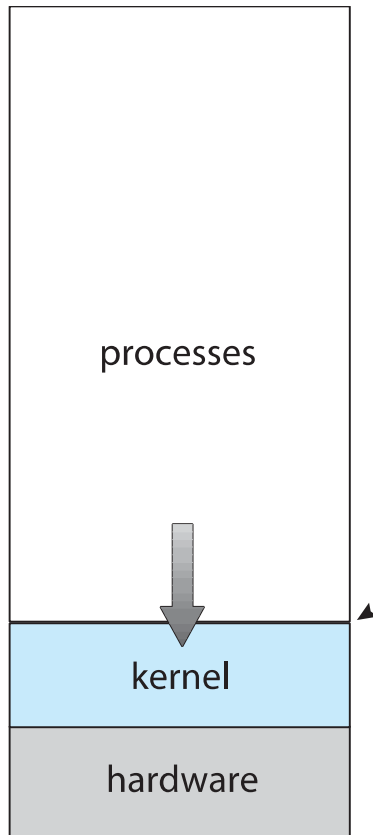
Computing Environments - Virtualization

- Use cases involve laptops and desktops running multiple OSES for exploration or compatibility
 - Apple laptop running Mac OS X host, Windows as a guest
 - Developing apps for multiple OSES without having multiple systems
 - QA testing applications without having multiple systems
 - Executing and managing compute environments within data centers
- VMM can run natively, in which case they are also the host
 - There is no general purpose host then (VMware ESX and Citrix XenServer)





Computing Environments - Virtualization



(a)

achine





Computing Environments – Cloud Computing

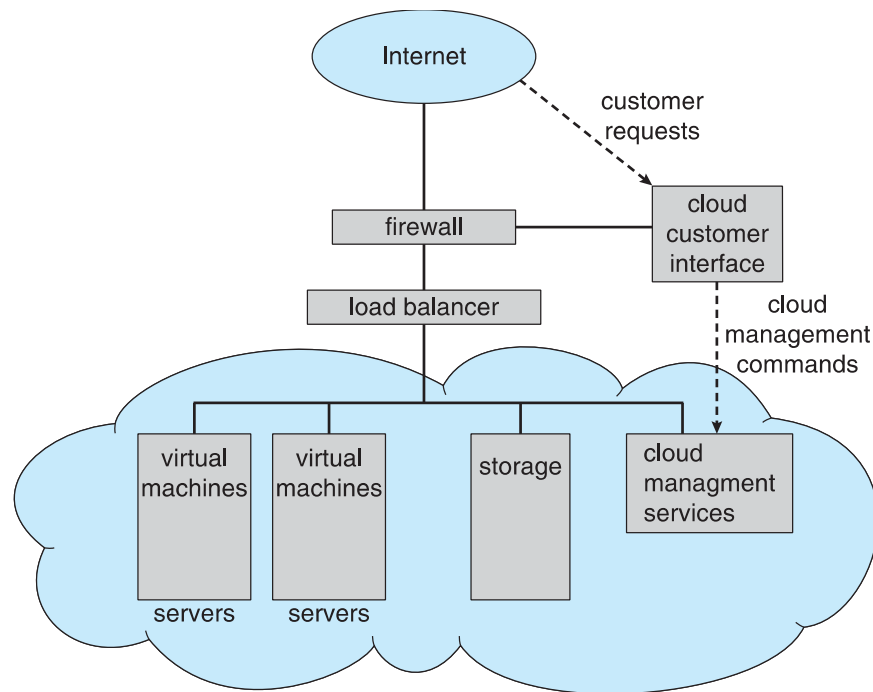
- Delivers computing, storage, even apps as a service across a network
- Logical extension of virtualization because it uses virtualization as the base for its functionality.
 - Amazon **EC2** has thousands of servers, millions of virtual machines, petabytes of storage available across the Internet, pay based on usage
- Many types
 - **Public cloud** – available via Internet to anyone willing to pay
 - **Private cloud** – run by a company for the company's own use
 - **Hybrid cloud** – includes both public and private cloud components
 - Software as a Service (**SaaS**) – one or more applications available via the Internet (i.e., word processor)
 - Platform as a Service (**PaaS**) – software stack ready for application use via the Internet (i.e., a database server)
 - Infrastructure as a Service (**IaaS**) – servers or storage available over Internet (i.e., storage available for backup use)





Computing Environments – Cloud Computing

- Cloud computing is the delivery of computing resources and services (like storage, servers, databases, software) over the Internet.
- Internet connectivity requires security like firewalls
- Load balancers spread traffic across multiple applications





Computing Environments – Real-Time Embedded Systems

- Real-time embedded systems most prevalent form of computers
 - Vary considerable, special purpose, limited purpose OS, **real-time OS**
 - Use expanding
- Many other special computing environments as well
 - Some have OSES, some perform tasks without an OS
- Real-time OS has well-defined fixed time constraints
 - Processing ***must*** be done within constraint
 - Correct operation only if constraints met





Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary **closed-source**
- Counter to the **copy protection** and **Digital Rights Management (DRM)** movement
- Started by **Free Software Foundation (FSF)**, which has “copyleft” **GNU Public License (GPL)**
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - <http://www.virtualbox.com>)
 - Use to run guest operating systems for exploration



End of Chapter 1

